

AGRIAI 2.0 | FORENSIC SOFTWARE EXAMINATION

Data, Authentication and Admin Internals

Schema, seed, durability, cookies, roles and runtime control plane

Volume 07

Prepared and authored solely by Thaddeus Nii Teiko Tagoe

Founder, designer and sole creator of AgriAI

Computer Science, University of Ghana

Repository examined: ThaddeusHackz/agriai-v2

Evidence commit: 991457272b7fec548e02c645c5593b930af5ea29

Examination date: 18 August 2026

Classification: Internal technical record

Technical basis

This document is grounded in direct source review, route enumeration, dependency audit, secret pattern scanning, strict TypeScript checking, ESLint, production compilation, PostgreSQL integration tests and local HTTP behavior tests. Live third party AI accounts were not available in the examination environment, so provider success is assessed from code paths and contracts rather than a claim of live production availability.

1. Database document schema

Collection or field	Stored content	Security or scale note
version	Seed schema version, currently 4	No migration framework beyond seed/load behavior
settings	Brand, sections, model, prompts, stats, contact	Runtime AI control plane
users	Name, email, bcrypt hash, role, timestamps	Admin and editor identities
sessions	Raw token, email, times, IP, user agent	Bearer-equivalent session material
chats	sessionId, title, mode, language, full messages	Unbounded farmer content
feedback	chat and message references, value, comment	Publicly appendable
prices	Crop, market, price, unit, date, trend	Public read, session write
knowledge	Question, answer, category, keyword list	Directly changes local AI answers
subscribers	Email, optional name and time	PII and CSV export
contacts	Name, email, subject, message, read flag	PII, no admin route visible in this snapshot
analytics	Visits, questions and counters	Can grow, public track input
meta	Seed timestamp	Used as price updatedAt even after price edits

2. Seed behavior

defaultDB() reads ADMIN_EMAIL, ADMIN_PASSWORD and ADMIN_NAME. If absent, it uses a known email, known password and generic name. It creates one bcrypt hash, 12 price rows, eight knowledge rows, empty record collections and default settings. resetDB() clears the cache, deletes db.json and immediately seeds again. If env remains absent, reset reintroduces the same known credential.

3. Local persistence algorithm

```
persist(db):  
  mkdir data recursively  
  write JSON.stringify(db, null, 2) to db.json.tmp  
  rename db.json.tmp to db.json  
  schedulePostgresSave(db)  
  on any local error: log and return
```

The temp rename gives atomic file replacement on the same filesystem. It does not provide a transaction across local disk and PostgreSQL. The in-process cache is the live object, and mutate() invokes the caller callback directly before persist(). There is no schema validation after reading JSON. A parse or read error causes a fresh seed and can mask recoverable corruption.

4. PostgreSQL mirror

lib/pg-store.ts creates a single table named agriai_state with integer primary key, JSONB doc and updated_at. Every save upserts id=1. schedulePostgresSave stores a pending document reference and waits 1,500 milliseconds. This reduces write frequency but means PostgreSQL is an asynchronous mirror, not the primary transaction coordinator.

```
CREATE TABLE agriai_state (  
  id INTEGER PRIMARY KEY,  
  doc JSONB NOT NULL,  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()  
)  
  
INSERT id=1, doc=$1::jsonb  
ON CONFLICT(id) DO UPDATE  
SET doc=EXCLUDED.doc, updated_at=now()
```

5. Hydration

1. instrumentation.register checks DATABASE_URL.

- 2. initPostgres creates the table.
- 3. hydrateFromPostgres checks whether local data/db.json exists.
- 4. If local data exists, PostgreSQL is not loaded.
- 5. If local data is absent, postgresLoad retrieves id=1.
- 6. A loaded document is assigned to cache and persisted locally again.

Consistency consequence

In a multi-instance deployment, each instance can hold a separate full document and each may overwrite id=1. Debounce and local files do not coordinate processes. A stale local file also wins over a newer PostgreSQL document at boot.

6. Passwords and sessions

Control	Implementation	Assessment
Password hash	bcrypt sync, cost 10	Reasonable prototype baseline, blocks plaintext storage
Session token	32 random bytes, hex	Strong entropy
Server storage	Database.sessions raw token	DB disclosure permits session replay
Cookie	HttpOnly, SameSite=Lax, Secure in production	Good base flags, no explicit Domain
Remember	Seven-day cookie	Matches seven-day server row
No remember	Eight-hour cookie	Server row remains seven days
Refresh	lastLogin update at most once per minute	Causes writes during admin use
Logout	Remove matching token and expire cookie	Correct direct invalidation

7. Login rate limiter

The limiter is a process-local Map keyed by the first x-forwarded-for value or x-real-ip. Five failures inside 15 minutes block further attempts. A successful login clears the IP entry. Old entries are pruned only when the map exceeds 5,000 keys. The control does not work consistently across multiple processes and relies on trusted proxy header sanitation.

8. Effective authorization matrix

Capability	Anonymous	Editor	Admin
Read public product	Yes	Yes	Yes
Use AI and media	Yes	Yes	Yes
Read chats and analytics	No	Yes	Yes
Read and export subscribers	No	Yes	Yes
Change prompts and model	No	Yes	Yes
Change content, appearance, knowledge and prices	No	Yes	Yes
Delete chats, feedback and subscribers	No	Yes	Yes
Force PostgreSQL sync	No	Yes	Yes
Manage users	No	No	Yes
Reset full database	No	No	Yes

9. Admin UI and server enforcement

AdminShell hides Users and Settings tabs from editors. The server protects user management and reset by role. The settings API itself remains available to editors, including action:sync and the AI prompt fields. This demonstrates why UI hiding and server authorization must be reviewed separately. An editor can call a route directly even if a tab is hidden.

10. Analytics mutations

- * trackVisit groups by ISO date and counts unique visits only by an in-process Set, so a restart can count the same visitor as unique again.
- * trackQuestion normalizes whitespace and stores the first 160 characters. It sorts by count and keeps 200 rows.
- * trackMessage increments before generation. totalMessages can diverge from the actual persisted chat messages if generation is aborted.
- * trackChat is called inside a mutate callback, and trackChat itself calls mutate. JavaScript permits this synchronous nesting, causing an extra full persist while the outer mutation is active.
- * Admin analytics wisely derives totalMessages and totalChats from chats instead of trusting all counters.

11. Data lifecycle recommendations

Entity	Recommended storage	Retention and control
settings and prompts	Versioned settings table	Permanent versions with approval and rollback
users	Users table	Permanent while active, MFA and password history policy
sessions	Hashed token session table or Redis	Short expiry, revoke on password and role changes
chats	Chat and message tables	Consent-based retention, default 90 days or less
analytics	Append-only bounded events or aggregate table	Minimize identifiers and expire raw events
subscribers and contacts	PII table with field protection	Purpose-based retention and deletion requests
audit	Append-only audit table	Restricted access and tamper evidence

Authorship and control statement

This volume was prepared, authored and issued solely under the name of Thaddeus Nii Teiko Tagoe. This volume explains the state engine and the exact authority behind every administrative action. The repository records identify Thaddeus Tagoe as the founder, sole designer, developer and maintainer of AgriAI.